Language Models for Law and Social Science

7. Encoder Models

David Zollikofer

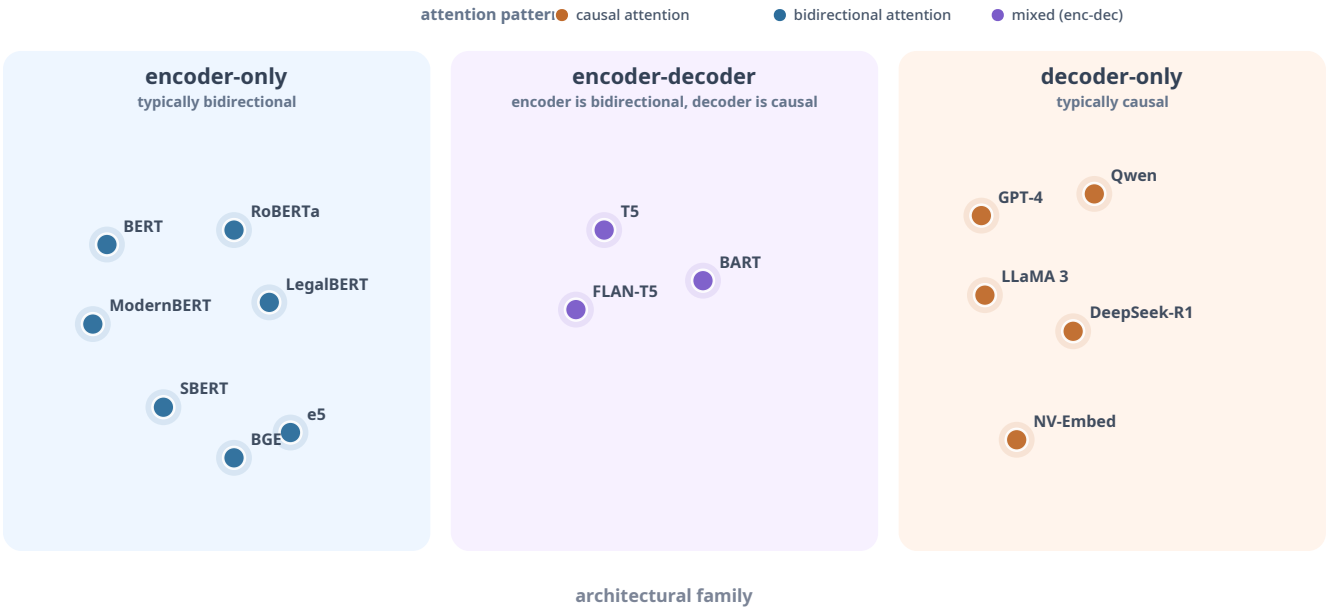
ETH Zürich

March 30, 2026

Outline

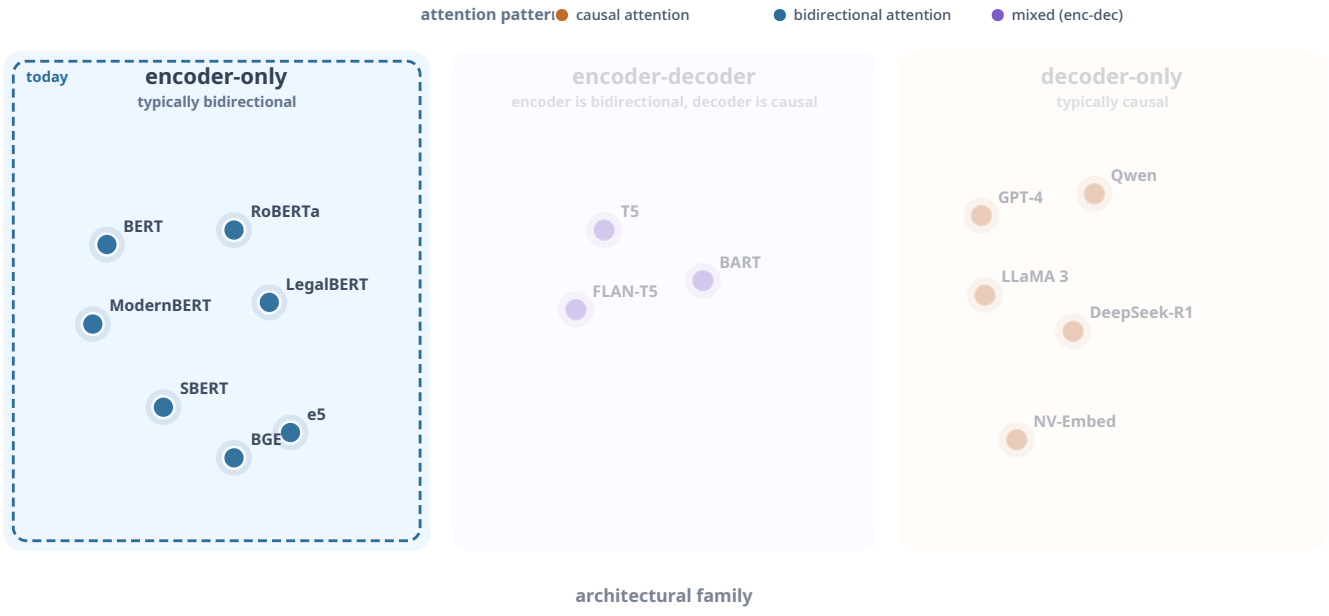
1. From self-attention to the encoder block
2. BERT and bidirectional pre-training
3. The encoder ecosystem
4. Fine-tuning for classification
5. Sentence embeddings: from SBERT to contrastive learning
6. Wrap-up

The Model Map



Color shows the dominant **attention pattern**: causal, bidirectional, or mixed. **Today:** the encoder bucket. **Next week:** decoder-only and encoder-decoder models.

Where We Are on the Model Map

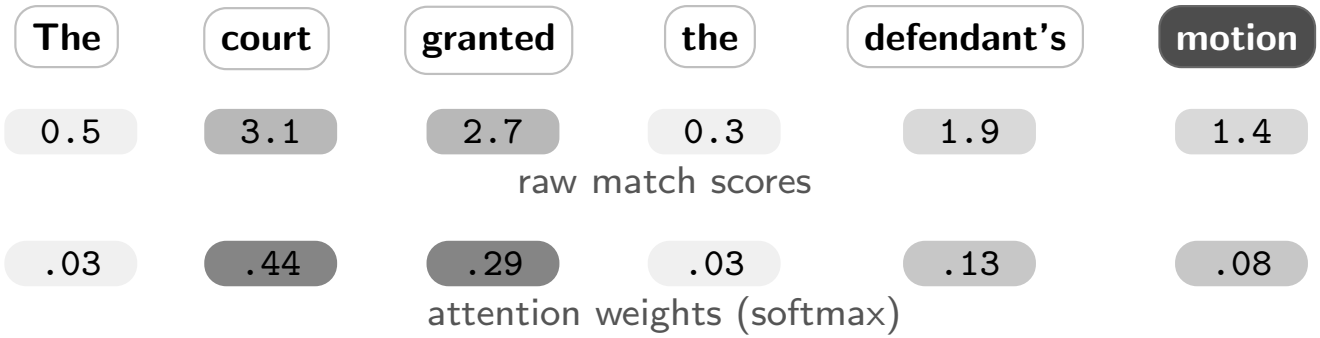


Today: encoder blocks, BERT, domain encoders, and embedding models. The attention-mask details come later; for now, this map just locates the main model families.

Recap: Self-Attention (Vaswani et al., 2017)

Pick one token — motion — and trace how it reads the sentence.

Each token already has a vector representation. To update **"motion"**, the model compares its vector to the vectors of the other tokens in the sentence.



$$\text{new representation of "motion"} \approx .03 \cdot \text{The} + \mathbf{.44} \cdot \text{court} + \mathbf{.29} \cdot \text{granted} + .03 \cdot \text{the} + .13 \cdot \text{defendant's} + .08 \cdot \text{motion}$$

Intuition: "motion" borrows most strongly from "court" and "granted," so its representation becomes **context-sensitive**.

This is the intuition, not the exact notation. The next slide shows the formal version with query, key, and value vectors.

Scaled Dot-Product Attention

Define self-attention as a function on a sequence of token vectors: it takes $(x_1, \ldots, x_n)$ and returns contextualized vectors $(h_1, \ldots, h_n)$.

$$\text{Attention}_{W^Q, W^K, W^V}(x_1, \ldots, x_n) = (h_1, \ldots, h_n)$$

For each token position, learned weights produce a query, key, and value vector:

$$q_i = W^Q x_i, \qquad k_i = W^K x_i, \qquad v_i = W^V x_i$$

$$\text{Attention}_{W^Q, W^K, W^V}(x_1, \ldots, x_n) = \left(\sum_{j=1}^{n} a_{1j} v_j, \ \ldots, \ \sum_{j=1}^{n} a_{nj} v_j \right), \quad a_{ij} = \text{softmax}_j\left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right)$$

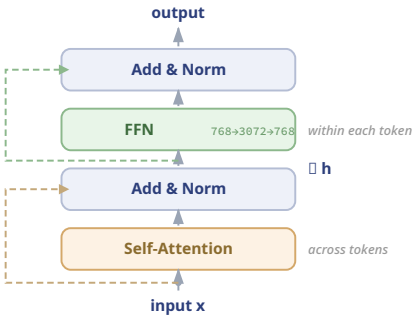
For token $i =$ motion, the score row compares q_{motion} to every key, the weight row is the softmax, and the output is the weighted sum of all value vectors.

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}, \qquad a_{ij} = \frac{e^{s_{ij}}}{\sum_{\ell=1}^{n} e^{s_{i\ell}}}, \qquad h_i = \sum_{j=1}^{n} a_{ij} v_j$$

The	court	granted	the	defendant's	motion
0.5	3.1	2.7	0.3	1.9	1.4
			$s_{\text{motion},j}$		
.03	.44	.29	.03	.13	.08
			$a_{\text{motion},j}$		

$$h_{\text{mot.}} \approx .03\, v_{\text{The}} + .44\, v_{\text{court}} + .29\, v_{\text{granted}} + .03\, v_{\text{the}} + .13\, v_{\text{def.}} + .08\, v_{\text{mot.}}$$

The Transformer Block (Vaswani et al., 2017; He et al., 2016; Ba et al., 2016)



Step 1 — attention + residual + normalize

$$h_i = \text{LayerNorm}\big(x_i + \text{MultiHeadAttn}(x_1, \ldots, x_n)_i\big)$$

Attention mixes information across tokens. The residual adds a shortcut path for the signal and the gradient, and LayerNorm normalizes across hidden dimensions with a learned scale and shift.

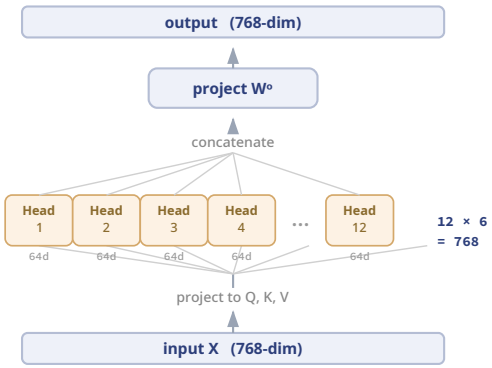
Step 2 — FFN + residual + normalize

$$o_i = \text{LayerNorm}\big(h_i + \text{FFN}(h_i)\big)$$

The FFN transforms each token independently: expand h_i, apply a nonlinearity, compress back to model dimension. Residual provides a shortcut; LayerNorm standardizes the output.

Attention communicates across tokens; the FFN computes within each token.

Multi-Head Attention (Vaswani et al., 2017)

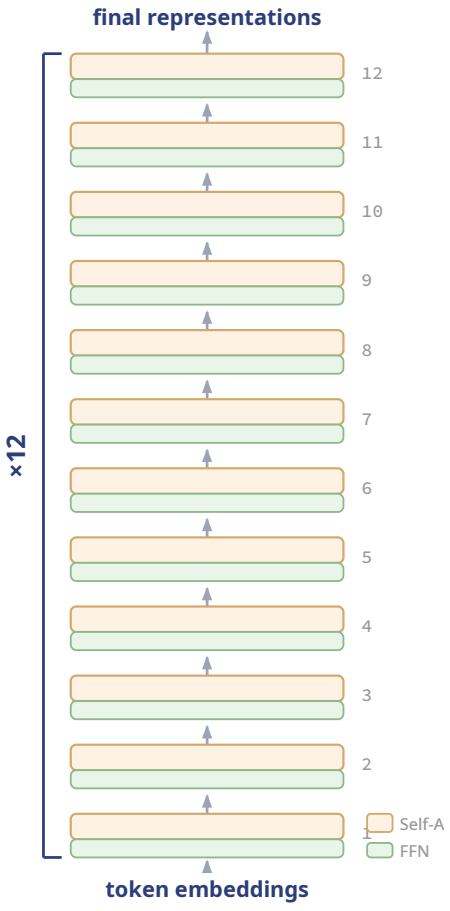


Input: $x_1, \ldots, x_n$ **Parameters:**
$W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$ for $i = 1, \ldots, h$;
$W^O \in \mathbb{R}^{hd_k \times d}$

1. $q_j^{(i)} = W_i^Q x_j, \quad k_j^{(i)} = W_i^K x_j, \quad v_j^{(i)} = W_i^V x_j$
2. $\text{head}_i = \text{Attention}\left(q^{(i)}, k^{(i)}, v^{(i)}\right)$
3. $\text{MultiHead}(x_1, \ldots, x_n) =$
   $\text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$

BERT-base: h=12, d_k=64, d=768. Multiple heads let the model capture several patterns in parallel.

Stacking Blocks into BERT



Each block is the same Transformer block from the previous slide, repeated **12 times**.

BERT-base has **110M parameters** in total.

Probing work suggests that different depths often emphasize different levels of linguistic structure (Tenney et al., 2019):

- **Lower layers:** surface features, word identity
- **Middle layers:** syntactic structure
- **Upper layers:** semantic relationships

These are average probing tendencies, not hard layer boundaries or guaranteed properties of the architecture.

Why Position Matters (Yun et al., 2020)

Pure self-attention is **permutation equivariant**: if we permute the input sequence $(x_1, \ldots, x_n)$, the outputs are permuted in the same way.

So without extra position information, self-attention has no built-in notion of absolute or relative order: permuting the input just permutes the output vectors in the same way.

For example:

- "The court rejected the motion"
- "The motion rejected the court"

These are permutations of the same tokens, so pure self-attention treats them as the same pattern up to permutation, not as two intrinsically ordered sequences.

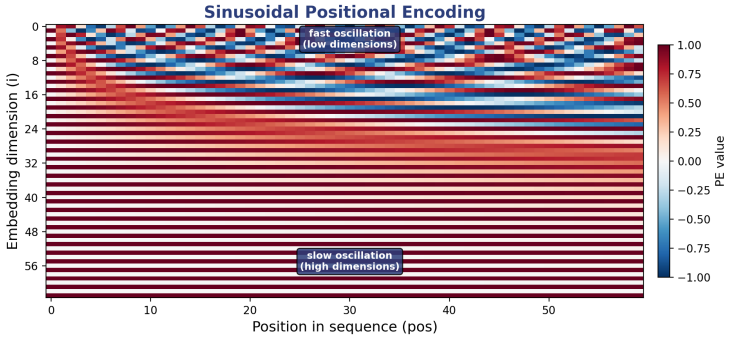
Positional Information

Three common ways to make order visible:

- **sinusoidal: deterministic waves across positions and dimensions**
- **learned: a trainable vector for each position**
- **rotary / RoPE: encode relative position directly in attention (Su et al., 2021)**

For BERT-style learned or sinusoidal position embeddings, order is added at the input:

$$x_i = \underbrace{e_i^{\text{token}}}_{\text{what word this is}} + \underbrace{e_i^{\text{position}}}_{\text{where it sits}}$$



Position is not inferred by the encoder block. It is added before the first block, so order is visible from the start.

The heatmap shows the original sinusoidal idea. RoPE instead injects position inside the attention computation rather than by summing an input embedding. **BERT specifically** uses **learned** position embeddings. Classic BERT also adds a separate segment embedding for sentence-pair inputs; that is a different signal from positional information.

What Goes Into BERT (Devlin et al., 2019)

Each token position starts as a constructed input vector. Token, position, and segment embeddings are learned lookup tables; training updates those vectors together with the rest of the model.

Tokenized input sequence

[CLS] the court upheld the appeal [SEP]

WordPiece (Wu et al., 2016) can split rarer words into subwords, e.g. `playing` → `play` + `##ing`. `[CLS]` is prepended to support sequence-level classification. `[SEP]` marks end of segment and separates A from B.

POSITION EMBEDDINGS

0 1 2 3 4 5 6

Each ID looks up a learned position embedding vector.

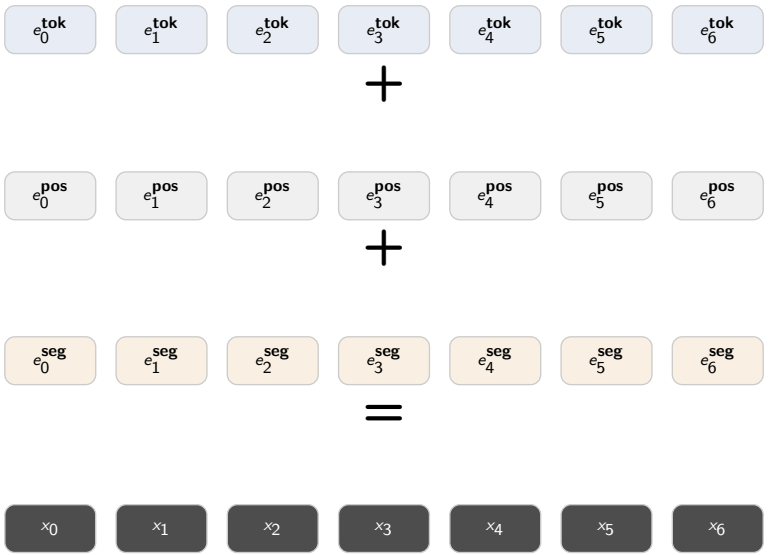
SEGMENT EMBEDDINGS

A A A A A A A

Likewise, **A** looks up a learned segment embedding vector.

Constructed input vectors

TOKEN EMBEDDINGS



$$x_i = e_i^{\text{tok}} + e_i^{\text{pos}} + e_i^{\text{seg}}$$

These constructed vectors are the inputs to encoder block 1. BERT-base then updates them through 12 bidirectional encoder blocks.

What Comes Out of BERT

The encoder's main output is a hidden vector at every token position.

Out of the encoder

$$(x_1, \ldots, x_n) \mapsto (h_1, \ldots, h_n)$$

- each hidden state belongs to a token **position**
- the two instances of **"the"** do not come out with the same vector
- **"bank"** in "river bank" and "central bank" also does not leave the encoder with the same vector

During MLM training only: unembedding
Take one chosen hidden state, e.g. h_{mask}, and map it back to vocabulary scores. This produces a distribution over vocabulary items, e.g.: **upheld, denied, filed, dismissed**.
The encoder returns vectors, not words. The MLM head **unembeds** one hidden vector by mapping it to one score per vocabulary item.

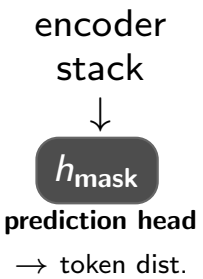
$$z = W_{\text{vocab}} h_{\text{mask}} + b, \qquad \hat{y} = \text{softmax}(z)$$

How BERT Is Trained

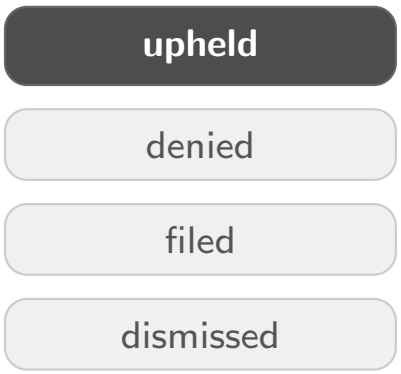
Masked input

[CLS] the court [MASK] the appeal [SEP]

BERT can use both left and right context to infer the missing token.
Masking rule: choose 15% of token positions for prediction.
Of those: 80% become `[MASK]`, 10% random token, 10% unchanged.
Target is always the original.



Prediction head



The head turns the masked-position hidden vector into vocabulary probabilities.

During masked-language-model pretraining, BERT selects some token positions for prediction; the encoder produces hidden vectors, and a temporary prediction head maps the masked-position vector back to vocabulary scores.
MLM loss = cross-entropy over masked positions.

$$z = \text{MLMHead}(h_{\text{mask}}), \qquad \hat{y} = \text{softmax}(z) \qquad \mathcal{L}_{\text{MLM}} = -\log \hat{y}_{w^\star}$$

In general: $\mathcal{L}_{\text{MLM}} = -\sum_{i \in M} \log p(x_i^\star \mid \tilde{x})$

Here $\tilde{x}$ denotes the corrupted input sequence after masking or random replacement. After pretraining, the prediction head is typically discarded; the encoder is reused for downstream tasks.

Pre-Train, Then Fine-Tune

Once BERT has learned this contextual fill-in-the-blank task, the paradigm is:

1. **Pre-train once** on massive unlabeled text
2. **Fine-tune cheaply** on a specific task

Why this mattered: labeled data are scarce, annotation is expensive, and domain corpora are noisy.

So instead of training a new model from scratch for every project, we adapt a general encoder to legal texts, legislative text, parliamentary speeches, or survey responses.

The Encoder Ecosystem

Three major directions after BERT:

- **better training recipe**: RoBERTa removes NSP and scales data (Liu et al., 2019)
- **smaller, faster model**: DistilBERT compresses BERT via distillation (Sanh et al., 2019)
- **domain-specific encoders**: LegalBERT, SciBERT, FinBERT, ClinicalBERT

And a more recent direction:

- **better long-context encoders**: ModernBERT with longer windows and modern training choices

The basic architecture stays, but the recipe and application domain change.

Domain Models Matter

Why not just use vanilla BERT?

Because specialized corpora change vocabulary and semantics:

- in legal text, "consideration" and "motion" have technical meanings
- in finance, "liability" and "guidance" mean something specific
- in biomedical text, abbreviations and entities are dense and domain-specific

Examples:

- **LegalBERT** (Chalkidis et al., 2020)
- **SciBERT** (Beltagy et al., 2019)
- **FinBERT** (Araci, 2019)
- **ClinicalBERT** (Alsentzer et al., 2019)
- **ModernBERT** (Warner et al., 2024)

Domain adaptation is often worth more than another clever downstream trick.

LEGAL-BERT: A Concrete Domain-Adaptation Test

(Chalkidis et al., 2020)

Like BioBERT and SciBERT, LEGAL-BERT asks how BERT should be adapted to legal text.
Research question: how should we adapt BERT for a legal sublanguage?
Three strategies: **(a)** generic **BERT** out of the box; **(b) further pre-train** on legal corpora; **(c) pre-train from scratch** with a legal vocabulary.
Pretraining corpus: $\sim$12 GB English legal text (EU, UK, ECHR, US laws, cases, contracts).
Evaluation tasks:

- **EURLEX57K:** assign EU legislative documents to EuroVoc concepts
- **ECHR-CASES:** classify case facts by Convention article violations
- **CONTRACTS-NER:** extract parties, dates, jurisdiction, amounts, etc.

Findings:

1. **Domain adaptation helps:** legal models usually beat vanilla BERT.
2. **No single recipe wins everywhere:** further pre-training and from-scratch both viable.
3. **Tuning matters:** hyperparameter search can change the ranking.

Method lesson: the baseline is not "BERT or no BERT," but which adaptation strategy fits the domain.

Fine-Tuning for Classification

Example: predict a document-level label such as *appeal affirmed vs. reversed* or *issue area*.

Fine-tuning adds a small classifier head on top of the final `[CLS]` vector:

$$\hat{y} = \text{softmax}(Wh_{[\text{CLS}]} + b)$$

What is learned during fine-tuning?

- a new task-specific classifier head
- usually the pretrained encoder weights as well, but with gentler updates than during pretraining

Why `[CLS]`? Because it can aggregate information from across the whole sequence and serve as a sequence-level representation.

Typical training setup:

- lower learning rate than pretraining
- small number of epochs
- validation-based stopping matters more than following one default recipe
- optional freezing of lower layers when labeled data are very scarce

The Embedding Problem (Reimers and Gurevych, 2019; Ethayarajh, 2019)

BERT outputs token vectors: $H = [h_1, \ldots, h_n] \in \mathbb{R}^{n \times d}$

An **embedding** is just a vector representation of text: a list of numbers meant to capture meaning.

So here, each token gets its own contextual embedding h_i, and the whole sentence produces a matrix H with one row per token.

But many tasks need **one vector per document**: semantic search, clustering, regression with text covariates.

Naive approaches — using `[CLS]` or mean pooling — don't work well:

- BERT was trained for **masked token prediction**, not for comparing sentences
- the representation space is **anisotropic**: vectors cluster in a narrow cone
- cosine similarity between raw BERT embeddings often underperforms simple baselines

The embedding space needs to be reshaped by a training signal that cares about similarity.

Training Sentence Embeddings

SBERT (2019): siamese bi-encoder with two weight-sharing BERT encoders, one per sentence. **Pool** = mean pooling over final hidden states (excluding padding).

$$u = \mathrm{Pool}(\mathrm{BERT}(s_1)), \qquad v = \mathrm{Pool}(\mathrm{BERT}(s_2))$$

- Original SBERT used **NLI sentence pairs** with softmax over entailment / contradiction / neutral:

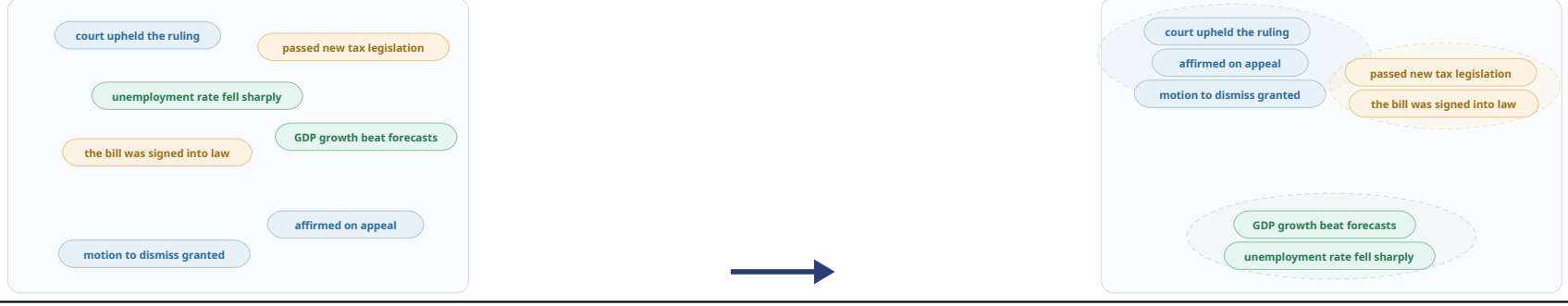
$$o = \mathrm{softmax}\big(W\,[u;\ v;\ |u-v|]\big), \qquad \mathcal{L}_{\mathrm{SBERT}} = -\sum_{c=1}^{3} y_c \log o_c$$

- At inference the classifier is discarded; similarity via cosine: $\mathrm{sim}(s_1, s_2) = \cos(u, v)$. Key gain: each sentence gets one reusable embedding.

Modern embedding models (SimCSE, E5, BGE) use **contrastive objectives**:

$$L_i = -\log \frac{\exp\big(\cos(q_i,\ d_i^{+})/\tau\big)}{\sum_{j=1}^{B} \exp\big(\cos(q_i,\ d_j)/\tau\big)}$$

Positive match moves closer; rest of batch acts as negatives. τ = temperature.



Modern Embedding Recipe (Zhang et al., 2025)

Better backbone + better loss

- **Example:** Qwen3 Embedding builds embeddings on top of a Qwen3 LLM backbone, using the final hidden state of the `[EOS]` token.
- Training optimizes an **improved InfoNCE-style contrastive loss**: pull the true match closer and push hard plus in-batch negatives away.

Really good data matters

1. Large-scale weak supervision from synthetic query-document pairs
2. Supervised fine-tuning on a smaller, higher-quality filtered dataset

The synthetic data are generated across multiple tasks, domains, and languages.

Takeaway: Modern embedding models are not just "BERT plus cosine." They rely on stronger backbones, better contrastive losses, and much better training data. Increasingly, strong embedding models are built on **causal LLM backbones**, not only on classic bidirectional encoders.

What Good Embeddings Enable

Once the embedding space is meaningful:

- **semantic search**: retrieve conceptually similar documents
- **clustering**: group cases, speeches, or policy documents by meaning
- **temporal analysis**: track how a concept shifts over time
- **regression / causal inference**: use embeddings as rich text covariates

Using Models in Practice

HuggingFace ecosystem

Model Hub: a large repository of pretrained models, searchable by task, domain, and language.

Three ways to use them:

```
# 1. Prototype
from transformers import pipeline
pipeline("sentiment-analysis")(
    "The court upheld the appeal.")

# 2. Fine-tune
from transformers import (AutoTokenizer,
    AutoModelForSequenceClassification)
tok = AutoTokenizer.from_pretrained(
    "bert-base-uncased")
model = AutoModelForSequenceClassification \
    .from_pretrained("bert-base-uncased",
    num_labels=2)

# 3. Embeddings
from sentence_transformers import (
    SentenceTransformer)
model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2")
model.encode(["The court upheld the appeal."])
```

Choosing a model

Task	Classification? Retrieval? Clustering?
Domain	Legal → LegalBERT; Biomedical → ClinicalBERT
Language	EN → BERT; Multilingual → XLM-R
Context	512 tok. (BERT) vs. 8192 (ModernBERT)

The bottleneck is not access to models. It is matching the model to the research design.

Today, many of these tasks can also be handled by general-purpose LLMs, but encoder models are still often cheaper, faster, and easier to deploy for prediction and retrieval.

Outlook

Next lecture: from encoder models to **causal language models** and **decoder-only transformers**.

We will ask:

- what **causal attention** means, and how it differs from bidirectional attention
- how decoder-only models generate text one token at a time
- why **scaling laws** matter for model behavior and capability
- what these models enable in social science: generation, coding assistance, summarization, extraction, simulation, and agentic workflows

References

- Ba et al. (2016). *Layer Normalization.*
- Beltagy et al. (2019). *SciBERT.*
- Chalkidis et al. (2020). *LEGAL-BERT.*
- Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers.*
- Ethayarajh (2019). *How Contextual are Contextualized Word Representations?*
- He et al. (2016). *Deep Residual Learning for Image Recognition.*
- Liu et al. (2019). *RoBERTa.*
- Reimers and Gurevych (2019). *Sentence-BERT.*
- Sanh et al. (2019). *DistilBERT.*
- Su et al. (2021). *RoFormer / RoPE.*
- Tenney et al. (2019). *BERT Rediscovers the Classical NLP Pipeline.*
- Vaswani et al. (2017). *Attention Is All You Need.*
- Warner et al. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference.*
- Wu et al. (2016). *Google Neural Machine Translation System.*
- Yun et al. (2020). *Are Transformers Universal Approximators of Sequence-to-Sequence Functions?*
- Zhang et al. (2025). *Qwen3 Embedding.*
- Araci (2019). *FinBERT.*
- Alsentzer et al. (2019). *ClinicalBERT.*